Day 2 — Production API Contracts for Applied AI Systems

Beginner-friendly summary

A production AI API is not merely a Python function exposed over HTTP. It is a **stable contract** between clients and a changing backend that may contain models, vector databases, agent workflows, queues, approval systems, and external providers.

Use:

- **Normal REST/JSON** when the operation finishes quickly and returns one result.
- **SSE streaming** when the client should see incremental output, such as LLM tokens or agent progress.
- **Asynchronous jobs** when work is slow, expensive, retryable, or must survive process restarts.
- **gRPC** mainly for strongly typed, high-throughput internal service-to-service communication.

Your contract should explicitly define validation, response schemas, status codes, error formats, idempotency behavior, pagination order, versioning, cancellation, and failure semantics.

---

1. REST, streaming, asynchronous jobs, and gRPC

Alternative	Best suited for	Selection criteria	Avoid when
REST with JSON	Predictions, CRUD, approval actions, job status	One request produces one reasonably fast result; browser/client compatibility matters	Results take minutes or require incremental output
SSE	LLM token streaming, agent progress, live job events	Server-to-client stream over HTTP; reconnect/resume semantics are useful	Client must continuously send messages over the same connection
Asynchronous job	Document ingestion, batch prediction, long forecasts, model evaluation	Work may exceed gateway timeout, needs durable retries, or consumes substantial resources	Operation is fast and clients need an immediate result
gRPC	Internal model serving, feature services, embedding services	Strong protobuf contracts, internal controlled clients, high request volume, unary or bidirectional streaming	Public browser-facing API or loosely coupled external consumers
WebSocket	Interactive bidirectional sessions	Both client and server must independently push messages	Communication is primarily request-response or one-way streaming

gRPC supports unary, server-streaming, client-streaming, and bidirectional-streaming RPC styles. SSE uses the `text/event-stream` media type and provides a one-way server-to-client event stream. (HTML Living Standard)

Practical selection rule

```
Can the operation reliably finish within the API timeout?
|
├── Yes
|   ├── Need incremental output? ── Yes ──> SSE
|   └── One final response? ──────── Yes ──> REST/JSON
|
└── No ──────────────────────────────────> 202 + Job resource

Internal, high-volume, strongly typed service call?
└── Consider gRPC
```

REST principles relevant to AI systems

A good REST API should:

1. Represent resources through stable URLs.
2. use HTTP methods consistently;
3. remain stateless between requests;
4. return representations rather than exposing internal implementation;
5. use HTTP status codes meaningfully;
6. avoid binding clients to a particular model provider.

AI APIs are often intentionally **REST-like rather than perfectly REST-pure**. Endpoints such as `/predict` and `/chat` resemble RPC commands, but remain reasonable when they expose stable request and response resources, identifiers, errors, and retry semantics.

For example:

```
POST /v1/predict
```

is pragmatic, while a more resource-oriented alternative is:

```
POST /v1/predictions
GET  /v1/predictions/{prediction_id}
```

For work accepted but not completed, HTTP `202 Accepted` is appropriate. It indicates acceptance for processing, not successful completion. (RFC Editor)

---

2. High-level architecture

```
 Client / SDK / UI
        |
        v
 API Gateway / Auth / Rate Limiting
        |
        v
+---------------- FastAPI ----------------+
```

```
| Correlation-ID middleware                |
| Authentication and dependencies          |
| Pydantic request validation              |
| Routers: predict/chat/docs/approvals     |
| Consistent problem-detail errors         |
+------------------+-----------------------+
                   |
          Application services
        /          |          \
       v           v           v
 Model Gateway   Job Queue   Approval Service
       |           |           |
 LLM/ML APIs    Workers     Transactional DB
 Vector DB      Object Store Idempotency Records
 Feature Store  Indexing     Audit / Outbox
```

The critical boundary is between the HTTP layer and application services. Routers should translate HTTP into application commands; they should not directly contain model-provider logic, SQL, vector-search orchestration, or agent implementation.

---

3. Recommended FastAPI application structure

```
app/
├── main.py
├── api/
│   ├── dependencies.py
│   └── v1/
│       ├── router.py
│       ├── predictions.py
│       ├── chat.py
│       ├── documents.py
│       ├── approvals.py
│       └── jobs.py
├── core/
│   ├── config.py
│   ├── errors.py
│   ├── middleware.py
│   ├── logging.py
│   └── security.py
├── schemas/
│   ├── common.py
│   ├── prediction.py
│   ├── chat.py
│   ├── forecasting.py
│   ├── documents.py
│   └── jobs.py
├── services/
│   ├── prediction_service.py
│   ├── chat_service.py
│   ├── document_service.py
│   └── approval_service.py
├── repositories/
```

```
│   ├── job_repository.py
│   ├── idempotency_repository.py
│   └── approval_repository.py
├── clients/
│   ├── model_gateway.py
│   ├── vector_store.py
│   └── object_store.py
├── workers/
│   └── document_ingestion.py
└── tests/
    ├── contract/
    ├── integration/
    └── unit/
```

FastAPI supports larger applications through `APIRouter`, reusable dependencies, ASGI middleware, and application lifespan handlers. The lifespan context is the preferred place to initialize and close shared resources such as model clients, database pools, HTTP clients, and queue connections. (FastAPI)

Layer responsibilities

Router

Responsible for:

- HTTP method and path;
- headers and query parameters;
- request and response models;
- status codes;
- authentication dependencies;
- translating domain exceptions into API errors.

It should not implement substantial business logic.

Service

Responsible for:

- prediction orchestration;
- retrieval and generation;
- agent execution;
- approval decisions;
- job creation;
- transaction boundaries.

Repository

Responsible for:

- persistence;
- optimistic concurrency;

- database queries;
- idempotency records;
- stable pagination.

Client

Responsible for:

- model-provider calls;
- vector database calls;
- timeouts and retries;
- provider-specific error translation;
- cancellation propagation.

---

4. Pydantic request and response contracts

Pydantic performs runtime validation and generates JSON Schema that FastAPI incorporates into OpenAPI.

Required, optional, and nullable fields

In Pydantic v2, `Optional[T]` or `T | None` means **nullable**, not necessarily optional. A default must be provided to make the field omittable. (Pydantic Docs)

Declaration	May be omitted?	May be `null`?
`name: str`	No	No
`name: str | None`	No	Yes
`name: str | None = None`	Yes	Yes
`name: str = "default"`	Yes	No
`items: list[str] = Field(default_factory=list)`	Yes	No

Example:

```
class ForecastRequest(BaseModel):
    series_id: str
    horizon: int = Field(ge=1, le=365)

    # Required and nullable. The key must appear.
    model_version: str | None

    # Optional and nullable. It may be omitted.
    timezone: str | None = None

    # Optional with non-null default.
    quantiles: list[float] = Field(default_factory=lambda: [0.1, 0.5, 0.9])
```

Field validation versus model validation

Use a field validator when the rule concerns one field:

```
@field_validator("content")
@classmethod
def reject_blank_content(cls, value: str) -> str:
    value = value.strip()
    if not value:
        raise ValueError("content must not be blank")
    return value
```

Use a model validator for relationships between fields:

```
@model_validator(mode="after")
def validate_time_range(self):
    if self.end_time <= self.start_time:
        raise ValueError("end_time must be later than start_time")
    return self
```

Pydantic supports field and model validators for both individual constraints and cross-field invariants. (Pydantic Docs)

Strict input design

For external requests, consider:

```
model_config = ConfigDict(extra="forbid")
```

Advantages:

- detects client spelling mistakes;
- prevents silently ignored inputs;
- makes contract violations visible.

Trade-off:

- a newer client sending an additive field to an older server will be rejected;
- this can complicate rolling deployments.

A common policy is:

- strict request validation for controlled first-party clients;
- carefully tolerant handling for long-lived external clients;
- response consumers must ignore unknown additive fields.

---

5. Status codes and consistent errors

Suggested status-code policy

Status	Meaning in this platform
`200 OK`	Prediction, chat response, job retrieval, completed action
`201 Created`	Approval record or newly created resource
`202 Accepted`	Document ingestion or long-running prediction queued
`204 No Content`	Successful cancellation or deletion with no response body
`400 Bad Request`	Invalid cursor or malformed business-level request
`401 Unauthorized`	No valid authentication
`403 Forbidden`	Authenticated but not permitted
`404 Not Found`	Job, document, model, or resource does not exist
`409 Conflict`	Idempotency conflict, request in progress, version conflict
`413 Content Too Large`	Upload exceeds accepted limit
`415 Unsupported Media Type`	Unsupported document type
`422 Unprocessable Content`	Structurally valid HTTP request fails field validation
`429 Too Many Requests`	Quota or rate limit exceeded
`500 Internal Server Error`	Unexpected application defect
`502 Bad Gateway`	Upstream model/provider returned an invalid failure
`503 Service Unavailable`	Model or dependency temporarily unavailable
`504 Gateway Timeout`	Upstream operation exceeded its time budget

Problem-details error envelope

RFC 9457 defines a machine-readable problem-details representation and supersedes the older RFC 7807. Its core fields include `type`, `title`, `status`, `detail`, and `instance`; applications may add extension fields. (RFC Editor)

Example:

```
{
  "type": "urn:problem:idempotency_key_reused",
  "title": "Idempotency key conflict",
  "status": 409,
  "detail": "The key was previously used with a different request payload.",
  "instance": "/v1/approvals",
  "code": "idempotency_key_reused",
  "correlation_id": "84ba7513ac744a2a9f9c533cb7606c55",
  "errors": [
    {
      "location": ["body", "expected_version"],
```

```
      "message": "Expected version 4 but current version is 5.",
      "code": "version_conflict"
    }
  ]
}
```

Correctness conditions

- The HTTP status and body `status` must agree.
- `code` should be stable and machine-readable.
- `detail` may change without breaking clients.
- Validation locations must not expose passwords, tokens, prompts, documents, or complete raw inputs.
- Internal exception messages and provider stack traces must not be returned.
- The correlation ID is diagnostic metadata, not authentication or authorization.

---

6. Endpoint contracts

`/v1/predict`

```
POST /v1/predict
Idempotency-Key: <client-generated-key>
Content-Type: application/json
```

Request:

```
{
  "model": "fraud-risk-model",
  "instances": [
    {
      "transaction_amount": 12000,
      "account_age_days": 520
    }
  ],
  "parameters": {
    "threshold": 0.7
  }
}
```

Response:

```
{
  "prediction_id": "UUID",
  "model": "fraud-risk-model",
  "model_version": "MODEL_VERSION",
  "predictions": [
    {
      "output": {
        "risk_score": 0.0,
```

```
        "decision": "PLACEHOLDER"
      }
    }
  ]
}
```

Contract decisions

- Batch input is a list even when only one instance is sent.
- Preserve output order relative to input order.
- Return the resolved model version, not merely the requested model alias.
- Avoid returning raw internal tensors or provider-specific responses.
- Either fail the entire batch atomically or define per-item errors explicitly.
- Do not silently mix results from different model versions.

`/v1/chat`

Request:

```
{
  "model": "assistant-model",
  "conversation_id": null,
  "messages": [
    {
      "role": "user",
      "content": "Explain the forecast anomaly."
    }
  ],
  "stream": false,
  "max_output_tokens": 512
}
```

Response:

```
{
  "response_id": "UUID",
  "conversation_id": "UUID",
  "model": "assistant-model",
  "status": "completed",
  "message": {
    "role": "assistant",
    "content": "..."
  },
  "usage": null
}
```

Senior-level considerations

- Decide whether the server or client owns conversation history.
- Do not accept trusted system prompts from untrusted clients.

- Version tool-call schemas independently.
- Define whether moderation happens before generation, after generation, or both.
- Clarify whether a response is persisted before being returned.
- Define what happens if retrieval succeeds but generation fails.
- Do not treat an LLM-generated approval statement as an actual approval.

Forecasting contract

A forecasting endpoint needs stronger temporal semantics than a normal prediction endpoint.

Suggested request:

```
{
  "series_id": "daily-demand-store-42",
  "model": "demand-forecast",
  "data_cutoff": "2026-08-05T23:59:59Z",
  "horizon": 14,
  "frequency": "P1D",
  "timezone": "Asia/Kolkata",
  "quantiles": [0.1, 0.5, 0.9],
  "known_future_covariates": [
    {
      "timestamp": "2026-08-07T00:00:00+05:30",
      "is_holiday": false
    }
  ]
}
```

Suggested response:

```
{
  "forecast_id": "UUID",
  "series_id": "daily-demand-store-42",
  "model_version": "MODEL_VERSION",
  "data_cutoff": "2026-08-05T23:59:59Z",
  "generated_at": "2026-08-06T09:30:00Z",
  "points": [
    {
      "timestamp": "2026-08-07T00:00:00+05:30",
      "p10": 100,
      "p50": 125,
      "p90": 155
    }
  ],
  "warnings": []
}
```

Correctness conditions:

- `data_cutoff` prevents accidental future-data leakage.

- Timestamps must include time-zone semantics.
- Forecast points must be unique and ordered.
- Quantiles should be monotonic: `p10 <= p50 <= p90`.
- The model version and data version must be traceable.
- Define how missing intervals and daylight-saving transitions are treated.
- Do not call lower and upper quantiles “confidence intervals” unless that is statistically correct for the model.

`/v1/documents`

Use multipart upload:

```
POST /v1/documents
Idempotency-Key: <key>
Content-Type: multipart/form-data
```

Parts:

```
file:       binary document
metadata:   JSON string
```

Response:

```
{
  "document_id": "UUID",
  "filename": "policy.pdf",
  "content_type": "application/pdf",
  "sha256": "...",
  "job": {
    "id": "UUID",
    "type": "document_ingestion",
    "status": "queued",
    "created_at": "...",
    "updated_at": "..."
  }
}
```

FastAPI’s `UploadFile` uses a spooled file abstraction, allowing larger files to move from memory to disk rather than requiring the complete file to remain in memory. (FastAPI)

Production validation should include:

- byte-size limits enforced while streaming;
- MIME allowlist;
- content sniffing rather than trusting only the declared MIME type;
- malware scanning;
- checksum calculation;
- tenant quota;

- encrypted object storage;
- document-level authorization;
- duplicate-content policy;
- archive and decompression-bomb protection.

`/v1/approvals`

Request:

```
{
  "resource_type": "financial_action",
  "resource_id": "transfer-482",
  "decision": "approve",
  "expected_version": 4,
  "comment": "Reviewed against the source records."
}
```

Response:

```
{
  "approval_id": "UUID",
  "resource_type": "financial_action",
  "resource_id": "transfer-482",
  "decision": "approve",
  "resource_version": 5,
  "decided_at": "2026-08-06T09:30:00Z"
}
```

Important design decisions:

- The approver identity must come from the authenticated principal, not the request body.
- Use optimistic concurrency through `expected_version`.
- Persist the approval, state transition, audit record, and outbox event in one transaction.
- An approval should authorize a specific immutable action payload, not a vague future action.
- Rejection should normally require a reason.
- Approval and action execution may be two separate operations.
- High-risk actions may require two-person approval.

`/v1/jobs/{id}`

Response:

```
{
  "id": "UUID",
  "type": "document_ingestion",
  "status": "running",
  "created_at": "...",
```

```
  "updated_at": "...",
  "result": null,
  "error": null
}
```

Recommended job state machine:

```
queued -> running -> succeeded
                  \-> failed
queued/running ---> cancelled
```

Avoid ambiguous states such as `done`, which do not distinguish success from failure.

---

7. Idempotency keys

An idempotency key allows a client to retry a non-idempotent request without accidentally executing the operation twice. The IETF idempotency-key draft describes using an `Idempotency-Key` request header for making operations such as `POST` fault-tolerant. (IETF)

Required algorithm

```
scope = tenant + endpoint + authenticated principal
fingerprint = hash(canonical request payload)

BEGIN TRANSACTION

record = SELECT idempotency_record
         WHERE scope = scope AND key = key
         FOR UPDATE

if no record:
    INSERT state=pending, fingerprint=fingerprint
    COMMIT
    execute operation

else if record.fingerprint != fingerprint:
    return 409 idempotency_key_reused

else if record.state == pending:
    return 409 request_in_progress
    or wait briefly according to documented policy

else if record.state == completed:
    return stored status code and stored response

on successful operation:
    atomically persist:
        business effect
        response
        idempotency state=completed
```

```
on transient failure:
    classify whether retry is safe
```

Financial action requirement

For a financial action, this is insufficient:

```
1. Execute money movement
2. Store idempotency result
```

The service could crash between steps.

Instead, the following must be within one transaction where possible:

```
idempotency record
+ ledger mutation
+ approval state
+ audit record
+ transactional outbox event
```

If the external payment provider owns the final side effect, propagate a stable idempotency key to that provider as well.

Key scope

Do not treat the key as globally unique without context. A safer identity is:

```
tenant_id
+ authenticated_client_id
+ endpoint
+ idempotency_key
```

Fingerprint

The same key with a different body must return `409 Conflict`.

Include business-relevant fields in the fingerprint, but exclude:

- correlation ID;
- request timestamp used only for tracing;
- transport-level metadata;
- fields normalized by the server.

Expiration

Document:

- how long keys remain valid;
- whether failed responses are cached;
- whether `4xx` responses are replayed;
- how concurrent requests are handled;

- maximum key length;
- whether keys can be queried by support staff.

---

8. Pagination, filtering, sorting, and stable cursors

Offset pagination

```
GET /v1/jobs?offset=1000&limit=20
```

Advantages:

- simple;
- permits direct page navigation.

Problems:

- increasingly expensive for large offsets;
- inserts or deletions can cause duplicates or missing items;
- unstable under concurrent changes.

Cursor pagination

```
GET /v1/jobs?limit=20&cursor=<opaque-value>
```

Use a stable ordering:

```
ORDER BY created_at DESC, id DESC
```

The cursor contains both values:

```
{
  "created_at": "2026-08-06T09:30:00Z",
  "id": "UUID"
}
```

The subsequent query is conceptually:

```
WHERE (created_at, id) < (:cursor_created_at, :cursor_id)
ORDER BY created_at DESC, id DESC
LIMIT 21;
```

Fetch `limit + 1` records to determine whether another page exists.

Correctness conditions

- Sorting must include a unique tie-breaker.
- Cursor fields must match the active sort.
- The cursor should be opaque to clients.

- Filters must remain the same across pages.
- Consider signing the cursor to detect tampering.
- The server should reject cursors from an incompatible schema or sort version.
- Never put sensitive filters or tenant identifiers into an unsigned readable cursor.

Example response:

```
{
  "items": [],
  "next_cursor": "eyJjcmVhdGVkX2F0IjoiLi4uIiwiaWQiOiIuLi4ifQ=="
}
```

---

9. API versioning and schema evolution

Recommended policy

Use URL major versions:

```
/v1/predict
/v2/predict
```

Do not create `/v2` for every additive change.

Usually backward-compatible:

- adding an optional response field;
- adding a new optional request field;
- adding a new endpoint;
- broadening a numeric limit carefully;
- adding a new error field while preserving stable error codes.

Potentially breaking:

- renaming or removing a field;
- changing field meaning;
- changing optional to required;
- changing nullable to non-nullable;
- changing units;
- changing timestamp interpretation;
- adding an enum value when clients use exhaustive matching;
- changing default sorting;
- changing idempotency scope or expiration;
- changing a synchronous operation into an asynchronous one.

Enum evolution trap

A client may write:

```
match job.status:
    case "queued":
        ...
    case "running":
        ...
    case "succeeded":
        ...
    case "failed":
        ...
```

Adding `"paused"` can break an exhaustive client even though the JSON schema change appears additive.

Client guidance should therefore be:

```
Handle known values explicitly.
Treat unknown values safely.
```

Deprecation lifecycle

1. Introduce replacement.
2. Mark the old operation deprecated in OpenAPI.
3. Send deprecation metadata.
4. Measure remaining consumers.
5. communicate the migration deadline;
6. send a sunset date;
7. remove only after the agreed support window.

The standardized `Deprecation` response header is defined by RFC 9745, and `Sunset` is defined by RFC 8594. (RFC Editor)

---

10. SSE streaming and cancellation

A typical event stream:

```
id: 1
event: message.delta
data: {"delta":"The "}

id: 2
event: message.delta
data: {"delta":"forecast "}

id: completed
event: message.completed
data: {"response_id":"...","status":"completed"}
```

Recommended event types:

```
message.started
message.delta
tool.started
tool.completed
message.completed
error
heartbeat
```

POST plus SSE nuance

Native browser `EventSource` establishes a URL-based event stream and is naturally suited to GET-style connections. For a `POST /v1/chat` that streams its response, browser clients commonly consume the response through streaming `fetch`, not native `EventSource`.

An alternative native-EventSource design is:

```
POST /v1/chat-sessions
-> returns stream_id

GET /v1/chat-streams/{stream_id}
Accept: text/event-stream
```

Cancellation algorithm

```
while generating:
    if client disconnected:
        cancel upstream model request
        close retrieval/database resources
        mark operation cancelled or retryable
        clear or transition idempotency reservation
        stop generating events
```

Starlette exposes `request.is_disconnected()` for detecting dropped clients in streaming or long-polling use cases. (Starlette)

Also catch task cancellation:

```
except asyncio.CancelledError:
    await model_client.cancel()
    raise
```

Important streaming failure condition

After response headers and stream bytes have been sent, the server cannot replace the response with an HTTP `500`.

Therefore, midstream failures must be represented as terminal events:

```
event: error
data: {
  "code": "provider_timeout",
```

```
  "correlation_id": "..."
}
```

Proxy considerations

For streams:

- disable proxy buffering;
- use heartbeats to prevent idle timeouts;
- bound total duration;
- enforce token and event limits;
- propagate deadlines to the model provider;
- avoid holding database transactions open;
- define reconnect and `Last-Event-ID` behavior.

---

11. Thought process for the practical implementation

Design decisions

1. Use `/v1` URL versioning.
2. Use Pydantic models with forbidden unknown fields for the demonstration.
3. place a correlation-ID middleware around every request;
4. convert validation and application failures to RFC 9457-style errors;
5. require idempotency keys on all side-effecting or costly POST operations;
6. use a request fingerprint to prevent key reuse with a different payload;
7. use synchronous JSON for `/predict` and non-streaming `/chat`;
8. use SSE when `chat.stream=true`;
9. use `202 Accepted` and a job resource for document ingestion;
10. use optimistic concurrency for approvals;
11. add `GET /v1/jobs` to demonstrate cursor pagination;
12. expose OpenAPI for contract review.

Pseudocode

```
APPLICATION START
    initialize database/model/queue clients
    initialize repositories
    expose routers

FOR EVERY REQUEST
    validate or generate correlation ID
    attach it to request context

    authenticate caller
    validate path/query/header/body using Pydantic

    if validation fails
```

```
        return application/problem+json

FOR IDEMPOTENT POST
    canonicalize relevant request
    calculate fingerprint
    reserve idempotency key

    if same key + different fingerprint
        return 409

    if same key is pending
        return 409 request_in_progress

    if same key is completed
        return stored response

    execute service operation

    atomically store result and completed idempotency record
    return response

FOR DOCUMENT UPLOAD
    validate declared media type
    stream chunks
    enforce byte limit
    calculate checksum
    persist object
    create queued job
    publish queue message
    return 202 with job URL

FOR CHAT STREAM
    reserve idempotency key
    call model gateway
    yield typed SSE events
    check disconnect between chunks

    on completion
        store final result
        emit message.completed

    on cancellation
        cancel provider request
        release resources
        transition idempotency state
```

12. Compact runnable FastAPI implementation

This is a **single-file learning implementation**. It demonstrates the contracts and control flow, but its stores and background task are in memory.

Install:

```
pip install fastapi uvicorn pydantic python-multipart pytest httpx
```

Save as `app/main.py`:

```
from __future__ import annotations

import asyncio
import base64
import hashlib
import json
import logging
import re
from collections.abc import Awaitable, Callable
from contextlib import asynccontextmanager
from dataclasses import dataclass
from datetime import datetime, timezone
from enum import StrEnum
from typing import Annotated, Any, Literal
from uuid import UUID, uuid4

from fastapi import (
    APIRouter,
    BackgroundTasks,
    Depends,
    FastAPI,
    File,
    Form,
    Header,
    Query,
    Request,
    UploadFile,
    status,
)
from fastapi.exceptions import RequestValidationError
from fastapi.responses import JSONResponse, StreamingResponse
from pydantic import BaseModel, ConfigDict, Field, field_validator,
model_validator

logger = logging.getLogger(__name__)
UTC = timezone.utc


#
# ---------------------------------------------------------------------------
# Base schemas and errors
#
# ---------------------------------------------------------------------------

class APIModel(BaseModel):
    model_config = ConfigDict(extra="forbid")


class FieldError(APIModel):
```

```
    location: list[str]
    message: str
    code: str


class ProblemDetail(APIModel):
    type_: str = Field(alias="type")
    title: str
    status: int
    detail: str
    instance: str | None = None
    code: str
    correlation_id: str
    errors: list[FieldError] | None = None


class APIError(Exception):
    def __init__(
        self,
        *,
        status_code: int,
        code: str,
        title: str,
        detail: str,
        errors: list[FieldError] | None = None,
    ) -> None:
        self.status_code = status_code
        self.code = code
        self.title = title
        self.detail = detail
        self.errors = errors
        super().__init__(detail)


#
# ---------------------------------------------------------------------------
# Prediction schemas
#
# ---------------------------------------------------------------------------

class PredictRequest(APIModel):
    model: str = Field(min_length=1, max_length=100)
    instances: list[dict[str, Any]] = Field(min_length=1, max_length=1000)
    parameters: dict[str, Any] | None = None

    @field_validator("instances")
    @classmethod
    def reject_empty_instances(
        cls,
        value: list[dict[str, Any]],
    ) -> list[dict[str, Any]]:
        if any(not instance for instance in value):
            raise ValueError(
                "each instance must contain at least one feature"
```

```
            )
        return value


class Prediction(APIModel):
    output: dict[str, Any]


class PredictResponse(APIModel):
    prediction_id: UUID
    model: str
    model_version: str
    predictions: list[Prediction]


#
# ---------------------------------------------------------------------------
# Chat schemas
#
# ---------------------------------------------------------------------------

class ChatMessage(APIModel):
    role: Literal["system", "user", "assistant", "tool"]
    content: str = Field(min_length=1, max_length=20_000)

    @field_validator("content")
    @classmethod
    def normalize_content(cls, value: str) -> str:
        value = value.strip()
        if not value:
            raise ValueError("content must not be blank")
        return value


class ChatRequest(APIModel):
    model: str = Field(min_length=1, max_length=100)
    conversation_id: UUID | None = None
    messages: list[ChatMessage] = Field(min_length=1, max_length=100)
    stream: bool = False
    max_output_tokens: int = Field(default=512, ge=1, le=8192)

    @model_validator(mode="after")
    def require_user_turn(self) -> "ChatRequest":
        if self.messages[-1].role != "user":
            raise ValueError("the last message must have role='user'")
        return self


class ChatResponse(APIModel):
    response_id: UUID
    conversation_id: UUID
    model: str
    status: Literal["completed"] = "completed"
    message: ChatMessage
```

```
    usage: dict[str, int] | None = None


#
# ---------------------------------------------------------------------------
# Job and document schemas
#
# ---------------------------------------------------------------------------

class JobStatus(StrEnum):
    QUEUED = "queued"
    RUNNING = "running"
    SUCCEEDED = "succeeded"
    FAILED = "failed"
    CANCELLED = "cancelled"


class JobRead(APIModel):
    id: UUID
    type: str
    status: JobStatus
    created_at: datetime
    updated_at: datetime
    result: dict[str, Any] | None = None
    error: ProblemDetail | None = None


class JobPage(APIModel):
    items: list[JobRead]
    next_cursor: str | None = None


class DocumentAccepted(APIModel):
    document_id: UUID
    filename: str
    content_type: str
    sha256: str
    job: JobRead


#
# ---------------------------------------------------------------------------
# Approval schemas
#
# ---------------------------------------------------------------------------

class ApprovalRequest(APIModel):
    resource_type: Literal[
        "agent_action",
        "forecast_override",
        "financial_action",
        "document_release",
    ]
    resource_id: str = Field(min_length=1, max_length=200)
```

```
    decision: Literal["approve", "reject"]
    expected_version: int = Field(ge=1)
    comment: str | None = Field(default=None, max_length=2000)

    @model_validator(mode="after")
    def require_rejection_comment(self) -> "ApprovalRequest":
        if (
            self.decision == "reject"
            and not (self.comment and self.comment.strip())
        ):
            raise ValueError(
                "comment is required when decision='reject'"
            )
        return self


class ApprovalResponse(APIModel):
    approval_id: UUID
    resource_type: str
    resource_id: str
    decision: str
    resource_version: int
    decided_at: datetime


#
# Idempotency
#

@dataclass
class IdempotencyRecord:
    fingerprint: str
    state: Literal["pending", "completed"]
    status_code: int | None = None
    response_body: dict[str, Any] | None = None


class IdempotencyStore:
    """
    In-memory demonstration only.

    Production:
    - Store records in a transactional database.
    - Scope keys by tenant, caller and endpoint.
    - Add TTL and cleanup.
    - Use a unique constraint and row locking.
    """

    def __init__(self) -> None:
        self._records: dict[str, IdempotencyRecord] = {}
        self._lock = asyncio.Lock()
```

```
    @staticmethod
    def _record_key(scope: str, key: str) -> str:
        return f"{scope}:{key}"

    async def begin(
        self,
        *,
        scope: str,
        key: str,
        fingerprint: str,
    ) -> tuple[Literal["new", "replay"], IdempotencyRecord | None]:
        record_key = self._record_key(scope, key)

        async with self._lock:
            record = self._records.get(record_key)

            if record is None:
                self._records[record_key] = IdempotencyRecord(
                    fingerprint=fingerprint,
                    state="pending",
                )
                return "new", None

            if record.fingerprint != fingerprint:
                raise APIError(
                    status_code=409,
                    code="idempotency_key_reused",
                    title="Idempotency key conflict",
                    detail=(
                        "The same Idempotency-Key was used with a "
                        "different request payload."
                    ),
                )

            if record.state == "pending":
                raise APIError(
                    status_code=409,
                    code="request_in_progress",
                    title="Request already in progress",
                    detail=(
                        "A request with this Idempotency-Key is still "
                        "being processed."
                    ),
                )

            return "replay", record

    async def complete(
        self,
        *,
        scope: str,
        key: str,
        fingerprint: str,
        status_code: int,
```

```
        response_body: dict[str, Any],
    ) -> None:
        record_key = self._record_key(scope, key)

        async with self._lock:
            self._records[record_key] = IdempotencyRecord(
                fingerprint=fingerprint,
                state="completed",
                status_code=status_code,
                response_body=response_body,
            )

    async def abort(self, *, scope: str, key: str) -> None:
        async with self._lock:
            self._records.pop(
                self._record_key(scope, key),
                None,
            )


def request_fingerprint(scope: str, payload: Any) -> str:
    canonical = json.dumps(
        {
            "scope": scope,
            "payload": payload,
        },
        sort_keys=True,
        separators=(",", ":"),
        default=str,
    ).encode("utf-8")

    return hashlib.sha256(canonical).hexdigest()


async def execute_idempotent(
    *,
    store: IdempotencyStore,
    scope: str,
    key: str,
    payload: Any,
    operation: Callable[[], Awaitable[tuple[int, BaseModel]]],
) -> JSONResponse:
    fingerprint = request_fingerprint(scope, payload)

    decision, record = await store.begin(
        scope=scope,
        key=key,
        fingerprint=fingerprint,
    )

    if decision == "replay":
        assert record is not None
        assert record.status_code is not None
        assert record.response_body is not None
```

```
        return JSONResponse(
            status_code=record.status_code,
            content=record.response_body,
        )

    try:
        status_code, result = await operation()

        body = result.model_dump(
            mode="json",
            by_alias=True,
            exclude_none=True,
        )

        await store.complete(
            scope=scope,
            key=key,
            fingerprint=fingerprint,
            status_code=status_code,
            response_body=body,
        )

        return JSONResponse(
            status_code=status_code,
            content=body,
        )

    except Exception:
        # Demonstration policy: cache only successful responses.
        # Production code must classify settled 4xx errors separately
        # from retryable 5xx or provider failures.
        await store.abort(scope=scope, key=key)
        raise


#
# ---------------------------------------------------------------------------
# Application lifecycle
#
# ---------------------------------------------------------------------------

@asynccontextmanager
async def lifespan(app: FastAPI):
    app.state.idempotency = IdempotencyStore()
    app.state.jobs: dict[UUID, JobRead] = {}
    app.state.resource_versions: dict[str, int] = {}

    # Production startup:
    # app.state.db = await create_database_pool()
    # app.state.model_client = ModelGateway(...)
    # app.state.queue = await connect_to_queue()

    yield
```

```
    # Production shutdown:
    # await app.state.queue.close()
    # await app.state.model_client.close()
    # await app.state.db.close()


app = FastAPI(
    title="Applied AI Platform API",
    version="1.0.0",
    lifespan=lifespan,
    openapi_url="/openapi.json",
    docs_url="/docs",
)

router = APIRouter(prefix="/v1")


#
--------------------------------------------------------------------------
# Correlation ID middleware
#
--------------------------------------------------------------------------

CORRELATION_ID_PATTERN = re.compile(
    r"^[A-Za-z0-9._:-]{1,128}$"
)


@app.middleware("http")
async def correlation_id_middleware(
    request: Request,
    call_next,
):
    incoming = request.headers.get("X-Correlation-ID", "")

    correlation_id = (
        incoming
        if CORRELATION_ID_PATTERN.fullmatch(incoming)
        else uuid4().hex
    )

    request.state.correlation_id = correlation_id

    response = await call_next(request)
    response.headers["X-Correlation-ID"] = correlation_id

    return response


#
--------------------------------------------------------------------------
# Error handlers
#
```

```
---------------------------------------------------------------------------

def problem_response(
    request: Request,
    *,
    status_code: int,
    code: str,
    title: str,
    detail: str,
    errors: list[FieldError] | None = None,
) -> JSONResponse:
    problem = ProblemDetail(
        type=f"urn:problem:{code}",
        title=title,
        status=status_code,
        detail=detail,
        instance=request.url.path,
        code=code,
        correlation_id=request.state.correlation_id,
        errors=errors,
    )

    return JSONResponse(
        status_code=status_code,
        media_type="application/problem+json",
        content=problem.model_dump(
            mode="json",
            by_alias=True,
            exclude_none=True,
        ),
    )


@app.exception_handler(APIError)
async def api_error_handler(
    request: Request,
    exc: APIError,
):
    return problem_response(
        request,
        status_code=exc.status_code,
        code=exc.code,
        title=exc.title,
        detail=exc.detail,
        errors=exc.errors,
    )


@app.exception_handler(RequestValidationError)
async def validation_error_handler(
    request: Request,
    exc: RequestValidationError,
):
    errors = [
```

```
        FieldError(
            location=[str(part) for part in error["loc"]],
            message=error["msg"],
            code=error["type"],
        )
        for error in exc.errors()
    ]

    return problem_response(
        request,
        status_code=422,
        code="validation_error",
        title="Request validation failed",
        detail="One or more request fields are invalid.",
        errors=errors,
    )


@app.exception_handler(Exception)
async def unhandled_error_handler(
    request: Request,
    exc: Exception,
):
    logger.exception(
        "Unhandled API error correlation_id=%s",
        request.state.correlation_id,
    )

    return problem_response(
        request,
        status_code=500,
        code="internal_error",
        title="Internal server error",
        detail="The request could not be completed.",
    )


#
# Dependencies
#

def get_idempotency_store(
    request: Request,
) -> IdempotencyStore:
    return request.app.state.idempotency


IdempotencyStoreDependency = Annotated[
    IdempotencyStore,
    Depends(get_idempotency_store),
]
```

```
IdempotencyKey = Annotated[
    str,
    Header(
        alias="Idempotency-Key",
        min_length=8,
        max_length=255,
    ),
]

ERROR_RESPONSES = {
    400: {"model": ProblemDetail},
    404: {"model": ProblemDetail},
    409: {"model": ProblemDetail},
    413: {"model": ProblemDetail},
    415: {"model": ProblemDetail},
    422: {"model": ProblemDetail},
    500: {"model": ProblemDetail},
}


#
--------------------------------------------------------------------------------
# Predict
#
--------------------------------------------------------------------------------

@router.post(
    "/predict",
    response_model=PredictResponse,
    responses=ERROR_RESPONSES,
    tags=["inference"],
)
async def predict(
    payload: PredictRequest,
    idempotency_key: IdempotencyKey,
    store: IdempotencyStoreDependency,
):
    async def operation() -> tuple[int, BaseModel]:
        # Conceptual stub. Replace with a model-gateway call.
        response = PredictResponse(
            prediction_id=uuid4(),
            model=payload.model,
            model_version="MODEL_VERSION_PLACEHOLDER",
            predictions=[
                Prediction(
                    output={"placeholder": True}
                )
                for _ in payload.instances
            ],
        )

        return status.HTTP_200_OK, response

    return await execute_idempotent(
```

```
        store=store,
        scope="POST:/v1/predict",
        key=idempotency_key,
        payload=payload.model_dump(mode="json"),
        operation=operation,
    )


#
--------------------------------------------------------------------------------
# Chat and SSE
#
--------------------------------------------------------------------------------

def format_sse(
    event: str,
    data: dict[str, Any],
    event_id: str | None = None,
) -> str:
    lines: list[str] = []

    if event_id:
        lines.append(f"id: {event_id}")

    lines.append(f"event: {event}")
    lines.append(
        "data: "
        + json.dumps(
            data,
            separators=(",", ":"),
            default=str,
        )
    )

    return "\n".join(lines) + "\n\n"


@router.post(
    "/chat",
    response_model=ChatResponse,
    responses={
        **ERROR_RESPONSES,
        200: {
            "description": (
                "JSON response or SSE when stream=true"
            ),
            "content": {
                "application/json": {
                    "schema": {
                        "$ref": (
                            "#/components/schemas/"
                            "ChatResponse"
                        )
                    }
```

```
                },
                "text/event-stream": {
                    "schema": {"type": "string"}
                },
            },
        },
    },
    tags=["chat"],
)
async def chat(
    request: Request,
    payload: ChatRequest,
    idempotency_key: IdempotencyKey,
    store: IdempotencyStoreDependency,
):
    conversation_id = (
        payload.conversation_id or uuid4()
    )
    response_id = uuid4()

    assistant_text = (
        "Conceptual placeholder response; "
        "connect your model gateway here."
    )

    scope = "POST:/v1/chat"

    def build_final_response() -> ChatResponse:
        return ChatResponse(
            response_id=response_id,
            conversation_id=conversation_id,
            model=payload.model,
            message=ChatMessage(
                role="assistant",
                content=assistant_text,
            ),
        )

    if not payload.stream:

        async def operation() -> tuple[int, BaseModel]:
            return (
                status.HTTP_200_OK,
                build_final_response(),
            )

        return await execute_idempotent(
            store=store,
            scope=scope,
            key=idempotency_key,
            payload=payload.model_dump(mode="json"),
            operation=operation,
        )
```

```
    fingerprint = request_fingerprint(
        scope,
        payload.model_dump(mode="json"),
    )

    decision, record = await store.begin(
        scope=scope,
        key=idempotency_key,
        fingerprint=fingerprint,
    )

    async def replay_stream():
        assert record is not None
        assert record.response_body is not None

        yield format_sse(
            "message.completed",
            record.response_body,
            event_id="completed",
        )

    if decision == "replay":
        return StreamingResponse(
            replay_stream(),
            media_type="text/event-stream",
            headers={
                "Cache-Control": "no-cache",
                "X-Accel-Buffering": "no",
            },
        )

    async def event_stream():
        try:
            tokens = assistant_text.split()

            for index, token in enumerate(tokens, start=1):
                if await request.is_disconnected():
                    await store.abort(
                        scope=scope,
                        key=idempotency_key,
                    )
                    return

                yield format_sse(
                    "message.delta",
                    {"delta": token + " "},
                    event_id=str(index),
                )

                await asyncio.sleep(0.02)

            completed = build_final_response()

            body = completed.model_dump(
```

```
                mode="json",
                exclude_none=True,
            )

            await store.complete(
                scope=scope,
                key=idempotency_key,
                fingerprint=fingerprint,
                status_code=200,
                response_body=body,
            )

            yield format_sse(
                "message.completed",
                body,
                event_id="completed",
            )

        except asyncio.CancelledError:
            await store.abort(
                scope=scope,
                key=idempotency_key,
            )
            raise

        except Exception:
            await store.abort(
                scope=scope,
                key=idempotency_key,
            )

            logger.exception("Streaming chat failed")

            yield format_sse(
                "error",
                {
                    "code": "stream_failed",
                    "detail": (
                        "The stream terminated before "
                        "completion."
                    ),
                    "correlation_id": (
                        request.state.correlation_id
                    ),
                },
            )

    return StreamingResponse(
        event_stream(),
        media_type="text/event-stream",
        headers={
            "Cache-Control": "no-cache",
            "X-Accel-Buffering": "no",
        },
```

```
    )


#
# ---------------------------------------------------------------------------
# Documents and asynchronous jobs
#
# ---------------------------------------------------------------------------

async def run_document_ingestion(
    app_instance: Any,
    job_id: UUID,
) -> None:
    """
    Demonstration only.

    A production implementation should publish a queue message and
    let a separate worker update the durable job record.
    """
    job: JobRead = app_instance.state.jobs[job_id]

    app_instance.state.jobs[job_id] = job.model_copy(
        update={
            "status": JobStatus.RUNNING,
            "updated_at": datetime.now(UTC),
        }
    )

    await asyncio.sleep(0.05)

    job = app_instance.state.jobs[job_id]

    app_instance.state.jobs[job_id] = job.model_copy(
        update={
            "status": JobStatus.SUCCEEDED,
            "updated_at": datetime.now(UTC),
            "result": {
                "indexed": True,
                "note": "conceptual in-process demo",
            },
        }
    )


@router.post(
    "/documents",
    status_code=status.HTTP_202_ACCEPTED,
    response_model=DocumentAccepted,
    responses=ERROR_RESPONSES,
    tags=["documents"],
)
async def upload_document(
    request: Request,
    background_tasks: BackgroundTasks,
```

```
    idempotency_key: IdempotencyKey,
    file: Annotated[
        UploadFile,
        File(description="Document to ingest"),
    ],
    store: IdempotencyStoreDependency,
    metadata: Annotated[
        str | None,
        Form(description="Optional JSON object"),
    ] = None,
):
    allowed_types = {
        "application/pdf",
        "text/plain",
        "text/markdown",
    }

    if file.content_type not in allowed_types:
        raise APIError(
            status_code=415,
            code="unsupported_media_type",
            title="Unsupported media type",
            detail=(
                f"Allowed content types: "
                f"{sorted(allowed_types)}"
            ),
        )

    metadata_object: dict[str, Any] = {}

    if metadata:
        try:
            parsed = json.loads(metadata)

            if not isinstance(parsed, dict):
                raise ValueError

            metadata_object = parsed

        except (json.JSONDecodeError, ValueError) as exc:
            raise APIError(
                status_code=422,
                code="invalid_metadata",
                title="Invalid document metadata",
                detail="metadata must be a JSON object.",
            ) from exc

    max_bytes = 10 * 1024 * 1024
    size = 0
    digest = hashlib.sha256()

    while chunk := await file.read(1024 * 1024):
        size += len(chunk)
```

```
        if size > max_bytes:
            raise APIError(
                status_code=413,
                code="document_too_large",
                title="Document too large",
                detail=(
                    "The maximum accepted document size "
                    "is 10 MiB for this demo."
                ),
            )

        digest.update(chunk)

    await file.close()

    sha256 = digest.hexdigest()

    operation_payload = {
        "filename": file.filename,
        "content_type": file.content_type,
        "sha256": sha256,
        "metadata": metadata_object,
    }

    async def operation() -> tuple[int, BaseModel]:
        now = datetime.now(UTC)

        job = JobRead(
            id=uuid4(),
            type="document_ingestion",
            status=JobStatus.QUEUED,
            created_at=now,
            updated_at=now,
        )

        document_id = uuid4()

        request.app.state.jobs[job.id] = job

        background_tasks.add_task(
            run_document_ingestion,
            request.app,
            job.id,
        )

        response = DocumentAccepted(
            document_id=document_id,
            filename=file.filename or "unnamed",
            content_type=(
                file.content_type
                or "application/octet-stream"
            ),
            sha256=sha256,
            job=job,
```

```
        )

        return status.HTTP_202_ACCEPTED, response

    return await execute_idempotent(
        store=store,
        scope="POST:/v1/documents",
        key=idempotency_key,
        payload=operation_payload,
        operation=operation,
    )


#
# ---------------------------------------------------------------------------
# Approvals
#
# ---------------------------------------------------------------------------

@router.post(
    "/approvals",
    status_code=status.HTTP_201_CREATED,
    response_model=ApprovalResponse,
    responses=ERROR_RESPONSES,
    tags=["approvals"],
)
async def create_approval(
    request: Request,
    payload: ApprovalRequest,
    idempotency_key: IdempotencyKey,
    store: IdempotencyStoreDependency,
):
    async def operation() -> tuple[int, BaseModel]:
        resource_key = (
            f"{payload.resource_type}:"
            f"{payload.resource_id}"
        )

        current_version = (
            request.app.state.resource_versions.setdefault(
                resource_key,
                1,
            )
        )

        if payload.expected_version != current_version:
            raise APIError(
                status_code=409,
                code="version_conflict",
                title="Resource version conflict",
                detail=(
                    f"Expected version "
                    f"{payload.expected_version}, "
                    f"but current version is "
```

```
                    f"{current_version}."
                ),
            )

        new_version = current_version + 1

        request.app.state.resource_versions[
            resource_key
        ] = new_version

        response = ApprovalResponse(
            approval_id=uuid4(),
            resource_type=payload.resource_type,
            resource_id=payload.resource_id,
            decision=payload.decision,
            resource_version=new_version,
            decided_at=datetime.now(UTC),
        )

        return status.HTTP_201_CREATED, response

    return await execute_idempotent(
        store=store,
        scope="POST:/v1/approvals",
        key=idempotency_key,
        payload=payload.model_dump(mode="json"),
        operation=operation,
    )


#
--------------------------------------------------------------------------------
# Stable cursor pagination
#
--------------------------------------------------------------------------------

def encode_cursor(job: JobRead) -> str:
    raw = json.dumps(
        {
            "created_at": job.created_at.isoformat(),
            "id": str(job.id),
        },
        separators=(",", ":"),
    ).encode("utf-8")

    return base64.urlsafe_b64encode(raw).decode("ascii")


def decode_cursor(
    cursor: str,
) -> tuple[datetime, str]:
    try:
        raw = base64.urlsafe_b64decode(
            cursor.encode("ascii")
```

```
        )

        payload = json.loads(raw)

        return (
            datetime.fromisoformat(payload["created_at"]),
            payload["id"],
        )

    except Exception as exc:
        raise APIError(
            status_code=400,
            code="invalid_cursor",
            title="Invalid pagination cursor",
            detail=(
                "The supplied cursor is malformed or "
                "no longer supported."
            ),
        ) from exc


@router.get(
    "/jobs",
    response_model=JobPage,
    responses=ERROR_RESPONSES,
    tags=["jobs"],
)
async def list_jobs(
    request: Request,
    limit: Annotated[
        int,
        Query(ge=1, le=100),
    ] = 20,
    cursor: Annotated[
        str | None,
        Query(),
    ] = None,
    status_filter: Annotated[
        JobStatus | None,
        Query(alias="status"),
    ] = None,
):
    items: list[JobRead] = list(
        request.app.state.jobs.values()
    )

    if status_filter:
        items = [
            job
            for job in items
            if job.status == status_filter
        ]

    items.sort(
```

```
        key=lambda job: (
            job.created_at,
            str(job.id),
        ),
        reverse=True,
    )

    if cursor:
        cursor_created_at, cursor_id = decode_cursor(
            cursor
        )

        items = [
            job
            for job in items
            if (
                job.created_at,
                str(job.id),
            )
            < (
                cursor_created_at,
                cursor_id,
            )
        ]

    window = items[: limit + 1]
    page_items = window[:limit]

    next_cursor = (
        encode_cursor(page_items[-1])
        if len(window) > limit
        else None
    )

    return JobPage(
        items=page_items,
        next_cursor=next_cursor,
    )


@router.get(
    "/jobs/{id}",
    response_model=JobRead,
    responses=ERROR_RESPONSES,
    tags=["jobs"],
)
async def get_job(
    request: Request,
    id: UUID,
):
    job = request.app.state.jobs.get(id)

    if job is None:
        raise APIError(
```

```
            status_code=404,
            code="job_not_found",
            title="Job not found",
            detail=f"No job exists with id '{id}'.",
        )

    return job


app.include_router(router)
```

Run:

```
uvicorn app.main:app --reload
```

OpenAPI:

```
GET /openapi.json
```

Interactive documentation:

```
GET /docs
```

---

13. Contract tests

FastAPI provides a test client through Starlette/httpx, enabling API tests without opening an external network socket. (FastAPI)

Save as `tests/test_contracts.py`:

```
import pytest
from fastapi.testclient import TestClient

from app.main import app


@pytest.fixture
def client():
    with TestClient(app) as test_client:
        yield test_client


def test_predict_contract(client: TestClient):
    response = client.post(
        "/v1/predict",
        headers={
            "Idempotency-Key": "predict-request-0001"
        },
        json={
            "model": "risk-model",
```

```
            "instances": [
                {"income": 100000, "loan_amount": 500000}
            ],
        },
    )

    assert response.status_code == 200

    body = response.json()

    assert body["model"] == "risk-model"
    assert "prediction_id" in body
    assert "model_version" in body
    assert len(body["predictions"]) == 1
    assert response.headers["X-Correlation-ID"]


def test_idempotent_replay_returns_same_result(
    client: TestClient,
):
    headers = {
        "Idempotency-Key": "predict-request-0002"
    }

    payload = {
        "model": "risk-model",
        "instances": [{"income": 100000}],
    }

    first = client.post(
        "/v1/predict",
        headers=headers,
        json=payload,
    )

    second = client.post(
        "/v1/predict",
        headers=headers,
        json=payload,
    )

    assert first.status_code == 200
    assert second.status_code == 200
    assert first.json() == second.json()


def test_idempotency_key_payload_conflict(
    client: TestClient,
):
    headers = {
        "Idempotency-Key": "predict-request-0003"
    }

    first = client.post(
```

```
        "/v1/predict",
        headers=headers,
        json={
            "model": "risk-model",
            "instances": [{"income": 100000}],
        },
    )

    conflict = client.post(
        "/v1/predict",
        headers=headers,
        json={
            "model": "risk-model",
            "instances": [{"income": 200000}],
        },
    )

    assert first.status_code == 200
    assert conflict.status_code == 409
    assert (
        conflict.headers["content-type"]
        == "application/problem+json"
    )
    assert (
        conflict.json()["code"]
        == "idempotency_key_reused"
    )


def test_validation_uses_problem_details(
    client: TestClient,
):
    response = client.post(
        "/v1/chat",
        headers={
            "Idempotency-Key": "chat-request-0001"
        },
        json={
            "model": "assistant-model",
            "messages": [],
        },
    )

    assert response.status_code == 422
    assert (
        response.headers["content-type"]
        == "application/problem+json"
    )

    body = response.json()

    assert body["code"] == "validation_error"
    assert body["status"] == 422
    assert body["correlation_id"]
```

```
    assert body["errors"]


def test_document_upload_returns_job(
    client: TestClient,
):
    response = client.post(
        "/v1/documents",
        headers={
            "Idempotency-Key": "document-request-0001"
        },
        files={
            "file": (
                "notes.txt",
                b"Example document",
                "text/plain",
            )
        },
        data={
            "metadata": '{"source":"contract-test"}'
        },
    )

    assert response.status_code == 202

    body = response.json()

    assert body["filename"] == "notes.txt"
    assert body["job"]["status"] == "queued"
    assert body["sha256"]


def test_approval_uses_optimistic_concurrency(
    client: TestClient,
):
    first = client.post(
        "/v1/approvals",
        headers={
            "Idempotency-Key": "approval-request-0001"
        },
        json={
            "resource_type": "financial_action",
            "resource_id": "transfer-42",
            "decision": "approve",
            "expected_version": 1,
        },
    )

    assert first.status_code == 201
    assert first.json()["resource_version"] == 2

    stale = client.post(
        "/v1/approvals",
        headers={
```

```
            "Idempotency-Key": "approval-request-0002"
        },
        json={
            "resource_type": "financial_action",
            "resource_id": "transfer-42",
            "decision": "approve",
            "expected_version": 1,
        },
    )

    assert stale.status_code == 409
    assert stale.json()["code"] == "version_conflict"


def test_required_paths_exist_in_openapi(
    client: TestClient,
):
    specification = client.get("/openapi.json").json()

    paths = specification["paths"]

    assert "/v1/predict" in paths
    assert "/v1/chat" in paths
    assert "/v1/documents" in paths
    assert "/v1/approvals" in paths
    assert "/v1/jobs/{id}" in paths

    schemas = specification["components"]["schemas"]

    assert "PredictRequest" in schemas
    assert "ChatRequest" in schemas
    assert "ProblemDetail" in schemas
```

---

14. OpenAPI-based contract review

OpenAPI is a language-independent API description that lets humans and tooling understand an HTTP API without inspecting its implementation. (OpenAPI Initiative Publications)

A senior-level review should check:

Request schemas

- Are required and nullable fields correct?
- Are minimum and maximum values documented?
- Are unknown fields handled intentionally?
- Are examples realistic but free of sensitive data?
- Are enum values evolvable?
- Are timestamps and units explicit?

Responses

- Does every successful response have a schema?
- Are `202`, `409`, `413`, `415`, `422`, `429`, and `5xx` cases documented?
- Do all errors use the same problem-detail envelope?
- Are streaming and JSON alternatives both documented?
- Is the job status model finite and explicit?

Compatibility automation

In CI:

```
generate new OpenAPI schema
        |
compare with main-branch schema
        |
classify changes:
    additive
    potentially breaking
    breaking
        |
require approval for breaking changes
        |
generate SDK and compile consumer tests
```

Important breaking-change checks:

- removed path;
- removed response field;
- newly required request field;
- narrower numeric range;
- changed field type;
- changed enum;
- changed status code;
- changed media type;
- changed nullability.

Do not rely only on schema-diff tooling. Some semantic changes, such as changing currency from rupees to paise or altering model-threshold meaning, may look schema-compatible while breaking clients.

---

15. Production trade-offs and failure modes

The reference implementation deliberately omits production infrastructure.

Demo component	Production replacement
In-memory idempotency dictionary	PostgreSQL/DynamoDB/Redis with documented durability and TTL
In-memory job dictionary	Transactional job table

FastAPI `BackgroundTasks`	Durable queue and separate worker
Placeholder model output	Model gateway with deadline, retries, circuit breaker and telemetry
In-memory resource version	Database row version or compare-and-swap
Base64 cursor	Signed/versioned opaque cursor
No authentication	OAuth/OIDC, service identity, scoped authorization
Process-local logging	Structured logs, traces and metrics
Single service state	Multi-replica-safe persistence

Major pitfalls

Running CPU-bound inference in the event loop

An `async def` endpoint does not make CPU-bound model inference non-blocking. Heavy local inference should run through:

- a separate model-serving process;
- GPU inference server;
- worker process;
- process pool for appropriate CPU workloads;
- asynchronous remote model client.

Using `BackgroundTasks` for durable work

In-process background tasks can disappear during:

- process crash;
- deployment;
- autoscaling;
- machine termination.

Use a durable queue for document ingestion, evaluation, batch forecasting, or long-running agent workflows.

Retrying every provider failure

Retry only when:

- the failure is classified as transient;
- the operation is idempotent;
- sufficient deadline remains;
- retry amplification is controlled;
- backoff and jitter are used.

Do not automatically retry:

- validation errors;
- authorization failures;
- model safety rejection;

- context-window violations;
- deterministic tool failure;
- non-idempotent external actions without a key.

Holding a database transaction during LLM generation

Never keep a database transaction open while waiting for:

- an LLM;
- a vector database;
- an external tool;
- a human approval.

Read required state, close the transaction, perform external work, and then persist through a new transaction using version checks.

Conflating model success with business success

An LLM successfully returning text does not mean:

- the tool executed;
- the forecast was accepted;
- the financial transaction completed;
- the agent action was approved;
- the document was indexed.

Represent these as separate states and resources.

Day 2 interview takeaway

A strong senior answer is:

> “I treat the API contract as a stable boundary around a changing AI platform. I use synchronous REST for bounded operations, SSE for incremental server output, and durable `202 + job` workflows for long-running work. Pydantic defines strict request and response schemas, while all errors use an RFC 9457-style envelope with stable codes and correlation IDs. Side-effecting operations use idempotency keys with request fingerprints and transactional persistence. Collection APIs use stable cursor pagination, and OpenAPI compatibility checks prevent accidental breaking changes. Model calls, persistence, and queue operations remain behind service and repository interfaces so providers and infrastructure can evolve without changing consumers.”

Day 2 DSA Add-on — Strings

Core string patterns

String interview problems are usually not about complicated string APIs. They test whether you can convert a textual requirement into one of a few reusable patterns.

Signal in the problem	Likely technique
“Same characters,” “anagram,” “occurrences”	Frequency counting
Ignore case, spaces, punctuation	Normalization
Compare from both ends	Left/right two pointers
Longest or shortest contiguous part	Sliding window
Subsequence rather than substring	Two pointers or dynamic programming
Repeated pattern or prefix/suffix	Hashing, prefix function, trie, or rolling hash
Many substring queries	Prefix counts, hashing, suffix structures

---

1. Frequency counting

Frequency counting maps each character to the number of times it appears.

```
text = "google"

frequency: dict[str, int] = {}

for character in text:
    frequency[character] = frequency.get(character, 0) + 1

print(frequency)
# {'g': 2, 'o': 2, 'l': 1, 'e': 1}
```

Typical uses:

- detecting anagrams;
- finding duplicate characters;
- comparing character inventories;
- maintaining a sliding window;
- checking whether a substring satisfies a required frequency.

For known lowercase English letters, an array can replace a dictionary:

```
counts = [0] * 26

for character in text:
    index = ord(character) - ord("a")
    counts[index] += 1
```

The array is usually faster and has predictable memory usage, but a dictionary is more flexible for Unicode or unknown character sets.

---

2. Normalization

Normalization removes representational differences that should not affect the answer.

Example requirement:

> Determine whether a sentence is a palindrome while ignoring case and non-alphanumeric characters.

A normalized representation might be:

```
normalized = "".join(
    character.lower()
    for character in text
    if character.isalnum()
)
```

However, creating a normalized copy requires additional memory. A two-pointer solution can often normalize characters while traversing the original string.

Important production consideration

`lower()` and `casefold()` are not identical.

```
text.casefold()
```

is more aggressive and better suited to case-insensitive Unicode comparison. Interview problems often assume ASCII unless stated otherwise, so clarify the expected character model.

---

3. Two-pointer string processing

Two pointers commonly appear in two forms.

Opposite-direction pointers

```
left  ->  "racecar"  <- right
```

Used for:

- palindrome checking;
- reversing;
- comparing prefixes and suffixes;
- skipping invalid characters.

Same-direction pointers

```
left ----> right
```

Used for:

- sliding windows;
- removing duplicates;
- partitioning;

- maintaining a valid substring.

A sliding window is a specialized two-pointer technique where `[left, right]` represents a contiguous substring.

---

4. Substring reasoning

A **substring** must be contiguous.

```
String:      "backend"
Substring:   "ack"
Not one:     "bed"  because those characters are not contiguous
```

A **subsequence** preserves order but does not need to be contiguous.

```
String:       "backend"
Subsequence:  "bed"
```

This distinction is an important recognition signal:

- longest substring generally suggests a window;
- longest subsequence often suggests dynamic programming or two independent pointers.

Medium problem — Longest Substring Without Repeating Characters

Problem statement

Given a string `s`, return the length of the longest substring containing no repeated characters.

Examples

```
Input:  "abcabcbb"
Output: 3

Explanation: "abc" has length 3.
```

```
Input:  "bbbbb"
Output: 1
```

```
Input:  "pwwkew"
Output: 3

Explanation: "wke" is a valid substring.
"pwke" is not a substring because it is not contiguous.
```

---

1. Recognition signals

The problem contains several strong signals:

- **“Longest”** means we are optimizing a range.
- **“Substring”** means the range must be contiguous.
- **“Without repeating characters”** defines a window-validity condition.
- When the window becomes invalid, removing characters from its left may restore validity.
- Therefore, use a **sliding window with two pointers**.
- Track characters using either:
  - a frequency map; or
  - the most recent index of each character.

The last-seen-index approach is especially efficient because `left` can jump directly past a duplicate.

---

2. Brute-force reasoning

Generate every possible substring and check whether it contains duplicate characters.

For every starting index:

1. Create an empty set.
2. Move the ending index forward.
3. Stop when a repeated character is found.
4. Record the largest valid length.

Brute-force pseudocode

```
maximum_length = 0

for start from 0 to n - 1:
    seen = empty set

    for end from start to n - 1:
        if string[end] is already in seen:
            break

        add string[end] to seen
        maximum_length = max(
            maximum_length,
            end - start + 1
        )

return maximum_length
```

This improved brute-force implementation is $O(n^2)$ rather than $O(n^3)$ because the set allows incremental duplicate detection.

A fully naive version that separately checks every generated substring can reach $O(n^3)$.

3. Optimized reasoning

Maintain a window:

```
s[left : right + 1]
```

The window must always contain unique characters.

Store the most recent index of each character:

```
last_seen[character] = index
```

When processing `s[right]`:

- If the character has not appeared inside the current window, extend normally.
- If it appeared at index `k` inside the current window, move:

```
left = k + 1
```

The important condition is:

```
last_seen[character] >= left
```

A character may have appeared earlier in the string but outside the current window. Such an occurrence must not move `left` backward.

A compact safe update is:

```
left = max(left, last_seen[character] + 1)
```

Window invariant

Before calculating the current length:

> The substring from `left` through `right` contains no repeated characters.

This invariant is the foundation of correctness.

When a duplicate appears:

1. Its previous occurrence divides the window.
2. Any valid window ending at `right` must start after that occurrence.
3. Moving `left` to `previous_index + 1` removes the conflict.
4. All other characters remain unique because the previous window was valid.

---

4. Edge cases

Consider these before coding:

Input	Expected result	Reason
`""`	`0`	No substring
`"a"`	`1`	One character
`"aaaa"`	`1`	Every extension repeats
`"abcdef"`	`6`	Entire string is unique
`"abba"`	`2`	`left` must never move backward
`" "`	`1`	A space is still a character
`"a b a"`	Depends on contract	Spaces count unless normalization is required
Unicode text	Depends on contract	Python iterates Unicode code points, not grapheme clusters

The `"abba"` case catches a common bug.

At the final `"a"`, its previous occurrence is outside the current window. Setting `left = previous_index + 1` without using `max` would incorrectly move `left` backward.

---

5. Complexity

Let `n` be the number of characters.

Brute force

- Time: `O(n²)`
- Space: `O(min(n, alphabet_size))`

Optimized sliding window

- Time: `O(n)`
- Space: `O(min(n, alphabet_size))`

Each character is processed once by `right`, and the dictionary lookup is average `O(1)`.

---

6. Optimized pseudocode

```
last_seen = empty map
left = 0
maximum_length = 0

for right from 0 to length(string) - 1:
    character = string[right]

    if character exists in last_seen:
        left = max(
```

```
            left,
            last_seen[character] + 1
        )

    last_seen[character] = right

    current_length = right - left + 1
    maximum_length = max(
        maximum_length,
        current_length
    )

return maximum_length
```

---

7. Python solution

```
def length_of_longest_substring(text: str) -> int:
    """Return the longest contiguous substring with unique characters."""
    last_seen: dict[str, int] = {}

    left = 0
    maximum_length = 0

    for right, character in enumerate(text):
        previous_index = last_seen.get(character)

        if previous_index is not None:
            left = max(left, previous_index + 1)

        last_seen[character] = right

        current_length = right - left + 1
        maximum_length = max(maximum_length, current_length)

    return maximum_length
```

Example usage

```
test_cases = [
    ("abcabcbb", 3),
    ("bbbbb", 1),
    ("pwwkew", 3),
    ("", 0),
    ("abcdef", 6),
    ("abba", 2),
]

for text, expected in test_cases:
    actual = length_of_longest_substring(text)
    print(
        f"{text!r}: actual={actual}, "
```

```
        f"expected={expected}"
    )
```

Dry run for `"abba"`

Initial state:

```
left = 0
maximum = 0
```

`right`	Character	Previous index	New `left`	Window	Maximum
0	`a`	None	0	`"a"`	1
1	`b`	None	0	`"ab"`	2
2	`b`	1	2	`"b"`	2
3	`a`	0	`max(2, 1) = 2`	`"ba"`	2

Without `max(left, previous_index + 1)`, the final step would incorrectly move `left` from `2` back to `1`.

Returning the actual substring

Sometimes the interviewer extends the requirement from length to substring.

```
def longest_unique_substring(text: str) -> str:
    last_seen: dict[str, int] = {}

    left = 0
    best_start = 0
    best_length = 0

    for right, character in enumerate(text):
        previous_index = last_seen.get(character)

        if previous_index is not None:
            left = max(left, previous_index + 1)

        last_seen[character] = right

        current_length = right - left + 1

        if current_length > best_length:
            best_start = left
            best_length = current_length

    return text[best_start : best_start + best_length]
```

Correctness condition

Update `best_start` only when a strictly longer window is found. Using `>=` would return the latest maximum-length substring instead of the earliest one.

That behavior is not inherently wrong, but it must match the specified tie-breaking contract.

---

8. Frequency-map alternative

Another solution stores character counts.

```
def length_of_longest_substring_with_counts(
    text: str,
) -> int:
    counts: dict[str, int] = {}

    left = 0
    maximum_length = 0

    for right, character in enumerate(text):
        counts[character] = counts.get(character, 0) + 1

        while counts[character] > 1:
            left_character = text[left]
            counts[left_character] -= 1
            left += 1

        maximum_length = max(
            maximum_length,
            right - left + 1,
        )

    return maximum_length
```

Comparison

Approach	Advantage	Trade-off
Last-seen index	Jumps `left` directly	Requires careful `max` logic
Frequency map	Generalizes to many window constraints	May move `left` one position at a time
Set	Simple mental model	Duplicate removal logic can be less direct

The frequency-map pattern generalizes better to problems such as:

- longest substring with at most `k` distinct characters;
- minimum window substring;
- finding anagrams;
- character-replacement windows.

---

9. Go comparison

```
package main

import "fmt"

func lengthOfLongestSubstring(text string) int {
    lastSeen := make(map[rune]int)

    left := 0
    maximumLength := 0
    position := 0

    for _, character := range text {
        if previousIndex, exists := lastSeen[character]; exists {
            candidateLeft := previousIndex + 1

            if candidateLeft > left {
                left = candidateLeft
            }
        }

        lastSeen[character] = position

        currentLength := position - left + 1

        if currentLength > maximumLength {
            maximumLength = currentLength
        }

        position++
    }

    return maximumLength
}

func main() {
    tests := []struct {
        input    string
        expected int
    }{
        {"abcabcbb", 3},
        {"bbbbb", 1},
        {"pwwkew", 3},
        {"", 0},
        {"abcdef", 6},
        {"abba", 2},
    }

    for _, test := range tests {
        actual := lengthOfLongestSubstring(test.input)

        fmt.Printf(
```

```
            "%q: actual=%d expected=%d\n",
            test.input,
            actual,
            test.expected,
        )
    }
}
```

Python versus Go considerations

Python

```
for right, character in enumerate(text):
```

Python strings behave as sequences of Unicode code points for typical iteration, and dictionary syntax is concise.

Go

Go strings are byte sequences. A loop using:

```
for index, character := range text
```

returns:

- `character` as a Unicode code point represented by `rune`;
- `index` as a **byte offset**, not the rune position.

That creates a subtle issue when calculating character-window lengths.

The Go solution above therefore maintains a separate `position` counter representing rune positions.

For ASCII-only interview constraints, a byte-oriented implementation can be simpler:

```
func lengthOfLongestASCIIString(text string) int {
    lastSeen := make(map[byte]int)

    left := 0
    maximumLength := 0

    for right := 0; right < len(text); right++ {
        character := text[right]

        if previousIndex, exists := lastSeen[character]; exists {
            if previousIndex+1 > left {
                left = previousIndex + 1
            }
        }

        lastSeen[character] = right
```

```
        currentLength := right - left + 1

        if currentLength > maximumLength {
            maximumLength = currentLength
        }
    }

    return maximumLength
}
```

At interview time, clarify:

> "Should I assume ASCII, lowercase English letters, or general Unicode?"

That decision affects both memory representation and indexing behavior.

---

10. Non-obvious design lessons

Why `left` only moves forward

The current window is already valid before adding the new rightmost character.

When a duplicate appears, starting before or at its previous occurrence remains invalid. Therefore, the earliest possible valid start is immediately after that previous occurrence.

Once a prefix has been excluded, no later operation should reintroduce it. Hence:

```
left is monotonically non-decreasing
```

This monotonic movement is one reason the algorithm is linear.

Why we update `last_seen` after adjusting `left`

The map should record the newest occurrence for future windows. The old occurrence is used to repair the current window, and then it is replaced.

Why this is a substring algorithm

At every step, the candidate is:

```
text[left : right + 1]
```

That is one continuous interval. We never skip internal characters.

---

11. Common interview mistakes

1. Confusing substring with subsequence.
2. Restarting the scan after finding a duplicate, causing $O(n^2)$ behavior.
3. Moving `left` backward in the `"abba"` case.

4. Returning the number of unique characters in the entire string instead of the longest unique window.
5. Using repeated slicing inside the loop, introducing unnecessary allocations.
6. Assuming lowercase ASCII without confirming constraints.
7. Forgetting that spaces and punctuation count unless normalization is requested.
8. Using Go byte indexes as Unicode character positions.
9. Updating the maximum before repairing an invalid window.
10. Claiming `O(1)` space without qualifying the alphabet assumption.

For a fixed ASCII alphabet, auxiliary space may be considered `O(1)`. For an unbounded character set, it is more accurately:

```
O(min(n, number of possible characters))
```

Interview-ready explanation

> "The words 'longest substring' suggest a sliding window because the answer must be contiguous. I maintain a valid window containing no duplicate characters and store each character's most recent index. When a repeated character occurs inside the current window, I move the left boundary directly after its previous occurrence. The left boundary never moves backward, so each character is processed once, giving `O(n)` time and `O(min(n, alphabet size))` space."